ARCHITECTURE & SYSTEM DESIGN

Argon Platform

Centralized Event Discovery, Inventory Reservation & Secure QR-Ticketing Architecture

TARGET MARKET
Cambodia (Urban Tech Events, Conferences & Festivals)

PAYMENT ARCHITECTURE
Digital Payment Gateway (QR & Webhook Settlement)

FRONTEND TECHNOLOGY
React (Modular Single Page Application & Mobile Web PWA)

BACKEND CORE
Fastify (Node.js) · PostgreSQL 16 · Redis 7

SECURITY ENGINE
HMAC-SHA256 Cryptographic One-Time QR Passwords

ADMISSION SLA
Sub-Second Camera Check-In with Full Offline Local Sync

SECTION 01 · STRATEGIC CONTEXT

1. Market Problem & Core Value Proposition

The Operational Breakdown in Cambodia Today

Most event organizers in Cambodia run ticket sales manually through social messaging platforms (Facebook Messenger and Telegram). This manual model causes severe operational breakdowns across the event lifecycle:

1. Inventory Overselling & Double-Booking

Organizers record seat orders manually on spreadsheets. When demand spikes, chat admins promise the same seats to different customers, causing embarrassing overselling and refund disputes.

2. Counterfeit & Forwarded QR Screenshots

Tickets sent as static photos get forwarded to multiple friends. Gate staff have no reliable mechanism to check if an image was already admitted at another door.

3. 45-Minute Door Verification Queues

Volunteers check bank transfer slips by hand on personal mobile phones at venue entrances. This manual checking creates massive lines and delays event starts.

4. Tedious Direct Bank Integration

Direct bank integration (such as direct Bakong NBC onboarding) requires extensive licensing, merchant agreements, and slow bureaucratic meetings that student and indie events cannot afford.

Argon's High-Level Architectural Solution

Argon replaces manual messaging with an automated web platform backed by a standard payment integration and sub-second gate scanning:

- **Automated Checkout with 10-Minute Hold:** When an attendee selects tickets, the system atomically locks the inventory. The buyer has 10 minutes to complete checkout before tickets automatically return to the public pool.
- **Standardized Digital Payment Gateway:** Rather than forcing complex direct central bank integrations, Argon connects to standard payment gateways (supporting local bank QR payments and digital checkout). Webhook callbacks settle transactions automatically without human slip verification.
- **Cryptographic One-Time QR Passes:** Every purchased ticket receives a unique signed token that invalidates permanently upon first scan. Screenshots cannot be shared for duplicate entry.
- **Sub-Second Door Check-In:** Staff scan tickets using any mobile browser in under one second, with full offline capability when venue Wi-Fi fails.

SECTION 02 · SYSTEM DESIGN

2. Subsystem Topology & Technology Stack Rationale

The platform separates user interfaces, core backend services, and storage systems into clear tiers to guarantee fast responses and reliable ticket handling.

The Four System Tiers

Tier 1: Attendee Discovery & Checkout Portal

Built with React. Provides a clean public catalog where attendees browse events, select ticket tiers, view seat maps, and complete digital payment checkouts.

Tier 2: Organizer Management Dashboard

Built with React. Enables event hosts to configure pricing tiers, track real-time sales curves, view live gate entrance numbers, and issue staff credentials.

Tier 3: Mobile Gate Check-In Client

Built with React (Mobile PWA). Lightweight progressive web app with camera barcode scanning. Runs on volunteer smartphones without app-store downloads and stores offline rosters.

Tier 4: Backend Core & Storage Engine

Fastify + PostgreSQL 16 + Redis 7. Handles inventory reservation locks, webhook payment settlement, ticket cryptographic token generation, and audit logging.

Technology Stack & Architectural Rationale

SUBSYSTEM	CHOSEN TECHNOLOGY	WHY CHOSEN (ARCHITECTURAL RATIONALE)	REJECTED ALTERNATIVE
Frontend Apps	React	Component-based modular architecture, universal ecosystem compatibility, fast client-side routing, and flexible deployment to any static CDN.	Heavy server-rendered frameworks with rigid deployment requirements.
Gate Scanner Client	React PWA (Barcode Detection API + Local Storage)	Zero friction for door volunteers: opens instantly in mobile browsers with native camera speeds and offline caching.	Native iOS/Android App (causes app-store review delays and install resistance for temp volunteers).
Backend Core API	Fastify (Node.js + Strict TypeScript)	High asynchronous I/O throughput, low memory footprint, fast JSON serialization, and compile-time schema safety.	Express / Python Django (higher memory consumption and slower concurrent request handling).
Relational Database	PostgreSQL 16	Strict ACID transactional guarantees, robust row-level concurrency locking for ticket capacity, and relational data integrity.	MongoDB / NoSQL (lacks strict multi-row transactional row locks for financial inventories).
Cache & Hold TTL	Redis 7	Sub-millisecond key expiration for 10-minute ticket holds and fast in-memory rate limiting against checkout bots.	Database cron polling (causes heavy table-scan lock contention on the database).
Payment Integration	Digital Payment Gateway (QR & Webhooks)	Automated webhook confirmation for instant ticket issuance; eliminates tedious direct central-bank onboarding meetings.	Manual bank slip upload (slow, error-prone, and prone to Photoshop slip fraud).

SECTION 03 · EXECUTION SEQUENCE DIAGRAMS

3. Core Request Flows: Ticket Reservation & Hold

Flow A: Ticket Reservation & Payment Gateway Generation

When an attendee initiates checkout, the backend locks inventory atomically and sets a 10-minute payment countdown:

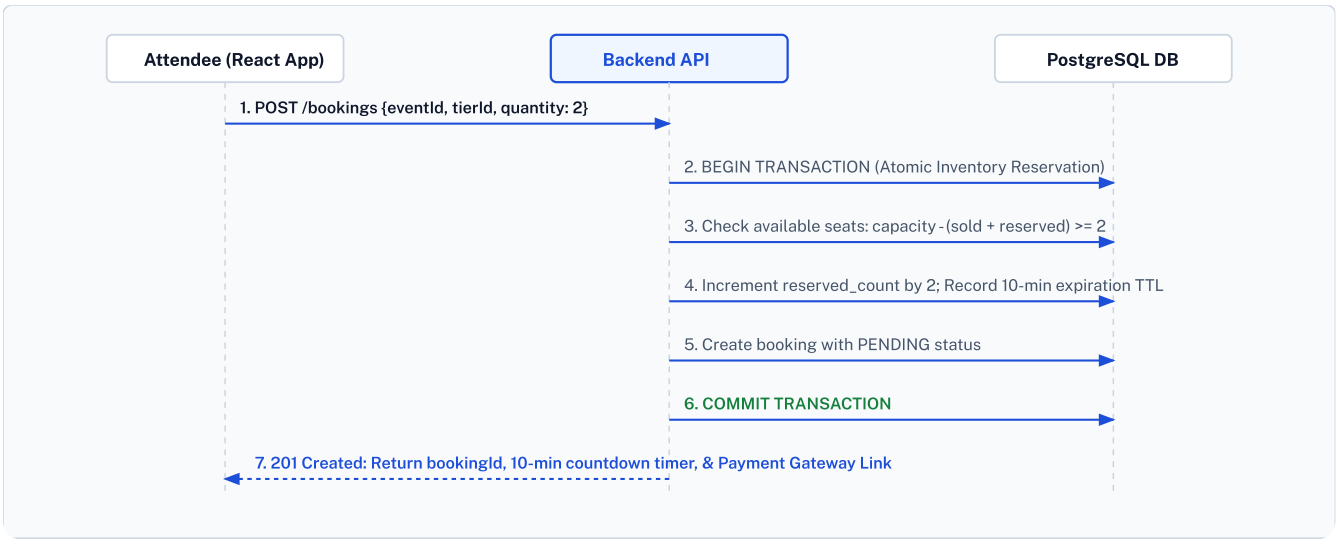


Figure 1: Sequence diagram for concurrent ticket inventory hold and payment initiation.

Key Mechanics & Failure Recovery

- **Atomic Hold Guarantee:** Inventory is locked within an ACID database transaction so two buyers hitting checkout at the same second cannot double-book the last seat.
- **Automatic Abandonment Recovery:** If the attendee closes their browser or fails to pay within 10 minutes, Redis expires the hold key and the backend decrements `reserved_count`, making tickets immediately available to other fans.

SECTION 03 · EXECUTION SEQUENCE DIAGRAMS

3. Core Request Flows: Payment & Pass Issuance

Flow B: Payment Settlement & Cryptographic Pass Issuance

When the payment gateway confirms receipt via webhook, the backend issues digital QR tickets:

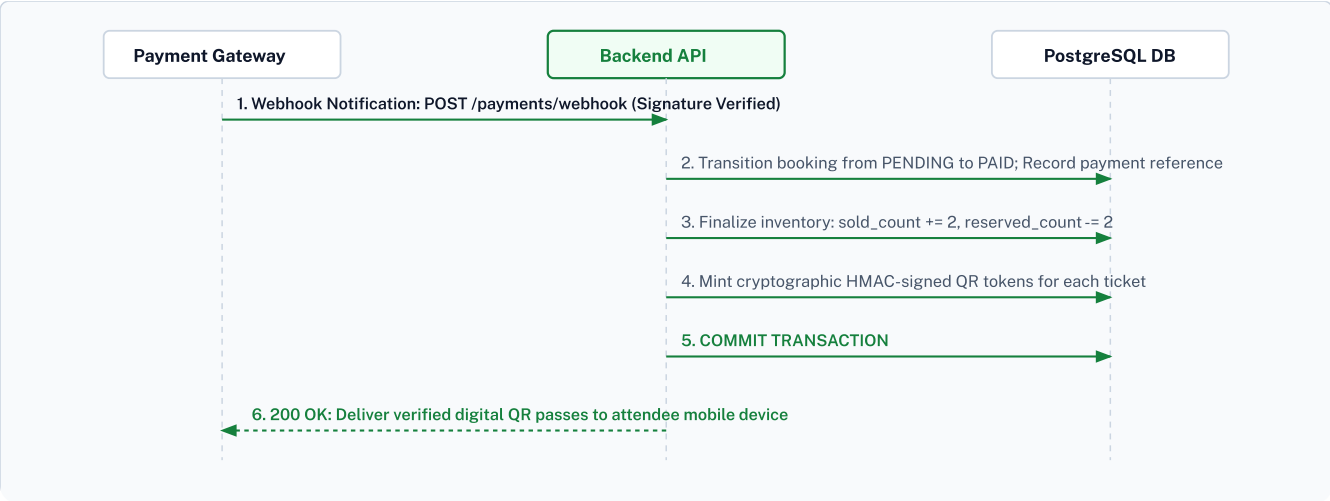


Figure 2: Sequence diagram for payment gateway webhook settlement and cryptographic pass generation.

Webhook Idempotency & Token Minting

- **Webhook Signature Verification:** Every payment callback includes a cryptographic signature from the gateway. Unsigned or replayed requests are rejected with zero database interaction.
- **Strict Idempotency:** If the gateway sends duplicate webhook events, the backend checks the transaction reference. If already marked `PAID`, the request returns `200 OK` immediately without double-minting tickets.

SECTION 03 · EXECUTION SEQUENCE DIAGRAMS

3. Core Request Flows: Door Check-In & Admission

Flow C: Gate Camera Scanning & Admission Check

When staff scan a ticket at the door, the system validates the cryptographic signature and marks the ticket used:

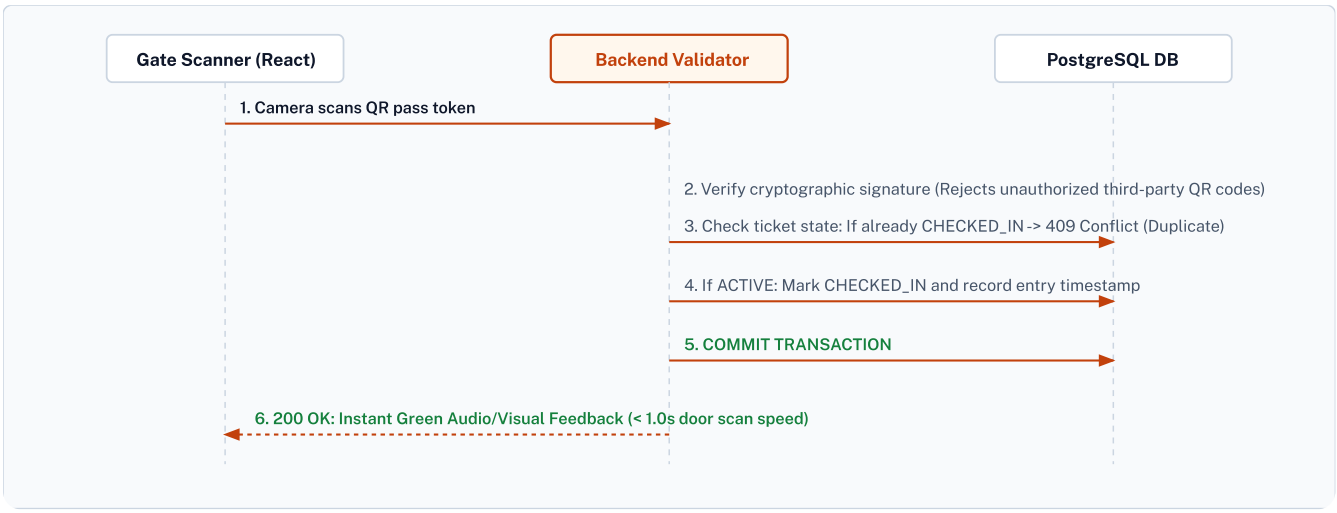


Figure 3: Sequence diagram for door ticket check-in and atomic duplicate prevention.

Sub-Second Door Admission Performance

- **Atomic State Transition:** The ticket transitions from `ACTIVE` to `CHECKED_IN` within an isolated row-level transaction, ensuring no two scanners can admit the same QR code at the same millisecond.
- **Immediate Operator Feedback:** The scanner emits distinct audio frequencies and visual indicators (Green chime for admission, Red buzz for duplicates or expired passes) to keep attendee queues moving continuously.

SECTION 04 · STATE MACHINES & DATA INTEGRITY

4. Entity Lifecycles & State Transition Models

1. The Booking Reservation Lifecycle

Bookings progress through strict states to ensure tickets are never stranded or double-sold:

STATE	TRIGGER EVENT	INVENTORY IMPACT	NEXT VALID STATE
`PENDING`	Attendee selects tickets and clicks Checkout.	Seats held in `reserved_count`; protected by a 10-minute Redis TTL.	`PAID` or `EXPIRED`
`PAID`	Payment gateway returns verified webhook confirmation.	`reserved_count` decrements; `sold_count` increments permanently.	`REFUNDED` (Organizer manual override only)
`EXPIRED`	10-minute timer lapses without payment.	`reserved_count` decrements back to public capacity automatically.	Terminal State
`CANCELLED`	User actively cancels or payment gateway reports payment failure.	Seats release immediately back to the available inventory pool.	Terminal State

2. The Individual Ticket Admission Lifecycle

Each individual ticket pass follows an irreversible state machine to eliminate door fraud:

PASS STATE	OPERATIONAL MEANING	SCANNER BEHAVIOR
`ACTIVE`	Ticket paid and valid for event entry.	Validates signature, changes state to `CHECKED_IN`, flashes green light.
`CHECKED_IN`	Ticket already scanned at the venue entrance.	Rejects scan immediately, flashes red light, and alerts staff with prior scan time.
`REVOKED`	Ticket refunded or invalidated by organizer.	Rejects scan immediately, flashes red light, and blocks venue admission.

SECTION 05 · SECURITY & OPERATIONAL TARGETS

5. Security Architecture & Operational SLAs

1. Why Static Screenshots Cannot Duplicate Entry

Standard event tickets fail because attendees screenshot their QR codes and send them to friends. Argon stops this with two layers of defense:

- **Cryptographic Signature:** Every QR code carries a server-signed HMAC-SHA256 token. Forged QR codes generated by third-party apps fail signature verification instantly.
- **Atomic State Invalidation:** When the first attendee passes through the gate, the ticket state becomes `CHECKED_IN`. If a second person arrives with a forwarded screenshot of the same QR code, the scanner buzzes red and reports the exact timestamp of earlier admission.

2. Offline Gate Redundancy (Network Failure Recovery)

Venue Wi-Fi and mobile networks frequently drop in convention halls and outdoor concert grounds. Argon's React PWA handles this automatically:

> **Local Cache Synchronization**
> Before event doors open, staff devices download an encrypted guest roster into the browser's local storage. When mobile connectivity drops, the scanner validates tickets against local storage with zero delay. As soon as connectivity returns, all check-in timestamps synchronize back to the central database.

3. Key Operational Performance Targets (SLAs)

PERFORMANCE METRIC	OPERATIONAL TARGET	PRACTICAL CONSEQUENCE
Gate Scan Speed	< 1.0 second per person	Prevents entrance bottlenecks and outdoor crowding chaos.
Checkout Hold TTL	10 minutes	Gives buyers sufficient time to complete payment without stranding tickets.
Allowable Duplicate Entry Rate	0 duplicate admissions allowed	Prevents unpaid attendee overcrowding and protects organizer revenue.
Scanner Device Requirements	Standard Mobile Browser (iOS / Android)	Zero rental fees for dedicated laser scanners; runs on volunteer smartphones.